

FINAL EXAM SOLUTION
(GRAPH MINING – SPRING 2026)

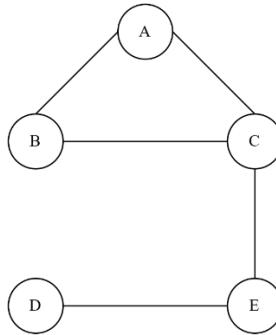
Question 1: (Graphlet Kernel) (10 points)

- a) Consider a small, undirected graph $G = (V, E)$ with 5 nodes where $V = A, B, C, D, E$, and $E = (A, B), (A, C), (B, C), (C, E), (E, D)$. Compute the graphlet count vector f_G for graphlets of size $k = 3$. (5 points)

1. g_1 : A path of length 2 (3 nodes, 2 edges)



2. g_2 : A triangle (3 nodes, 3 edges)



- b) How does the choice of graphlet size k dictate the trade-off between the expressive power of the Graphlet Kernel and its computational complexity? (5 points)

Solution:

a)

	$\{A, B, C\}$
	$\{A, C, E\}, \{B, C, E\}, \{C, E, D\}$

f_G is represented as (Count of g_1 , Count of g_2):

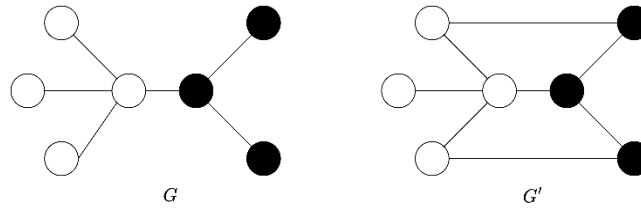
$$f_G = (3, 1)$$

- b) **Expressive Power:** As k increases, the kernel captures richer, higher-order structural invariants (such as complex network motifs, larger cliques, and intricate neighborhood topologies). This dramatically improves the kernel's resolution, allowing it to distinguish between structurally similar but strictly non-isomorphic graphs that a lower k would incorrectly map to the same feature representation.

Computational Complexity: The time required to exhaustively enumerate and count all subgraphs of size k scales exponentially. For a graph with n nodes, the worst-case time complexity is firmly bounded by $O(n^k)$. Furthermore, the dimensionality of the feature space explodes, as the number of possible unique, non-isomorphic graphlets grows factorially with k .

Question 2: (Shortest Path Kernel) (5 points)

Consider the following two small undirected graphs with node labels (black and white)



Given the shortest-path graph kernel is:

$$k_{\text{shortest paths}}(S_1, S_2) = \sum_{e_1 \in E_1} \sum_{e_2 \in E_2} k_{\text{walk}}^{(1)}(e_1, e_2)$$

- Compute all-pairs shortest path lengths for both graphs and compute the shortest path kernel value $K_{SP}(G, G')$. (3 points)
- What are the main theoretical and practical limitations of the shortest path kernel? (2 points)

Solution:

a)

$$\phi_{\text{path}}(G) = [3 \quad 2 \quad 1 \quad 3 \quad 1 \quad 5 \quad 0 \quad 0 \quad 6]$$

$$\phi_{\text{path}}(G') = [3 \quad 2 \quad 3 \quad 3 \quad 1 \quad 5 \quad 0 \quad 0 \quad 4]$$

$$k_{SP}(G, G') = (3 \times 3) + (2 \times 2) + (1 \times 3) + (3 \times 3) + (1 \times 1) + (5 \times 5) + (0 \times 0) + (0 \times 0) + (6 \times 4)$$

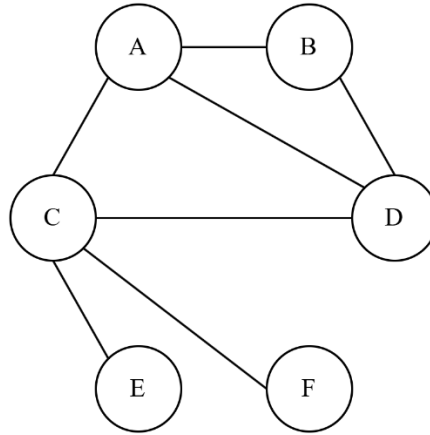
$$k_{SP}(G, G') = 75$$

- b) High computational cost:** Computing all-pairs shortest paths via Floyd-Warshall is $O(n^3)$ per graph

Scalability issues: Memory-intensive for dense graphs (shortest-path "graphs" can become dense). Not suitable for very large or streaming graphs.

Question 3: (Weiseiler-Lehman Kernel) (10 points)

- a) Perform one iteration (color refinement) of the 1-WL test for the given graph. Initialize all nodes with the color “1”. Show the multi-set of neighborhood colors for each node, the aggregated string, and assign new hashed colors. (5 points)



- b) Provide an example of a simple graph structure that the 1-WL test is provably incapable of detecting or counting. (5 points)

Solution:

- a) We first determine the adjacency list for each node to identify their neighborhoods $\mathcal{N}(v)$:

$$\mathcal{N}(A) = \{B, C, D\}$$

$$\mathcal{N}(B) = \{A, D\}$$

$$\mathcal{N}(C) = \{A, D, E, F\}$$

$$\mathcal{N}(D) = \{A, B, C\}$$

$$\mathcal{N}(E) = \{C\}$$

$$\mathcal{N}(F) = \{C\}$$

Initial State, all nodes are initialized with the same feature/color:

$$c^{(0)}(v) = 1 \quad \forall v \in \{A, B, C, D, E, F\}$$

First Iteration

Step 1: Compute neighborhood color multisets

Node	Degree	Neighborhood color multiset
A	3	$\{1, 1, 1\}$
B	2	$\{1, 1\}$
C	4	$\{1, 1, 1, 1\}$
D	3	$\{1, 1, 1\}$
E	1	$\{1\}$
F	1	$\{1\}$

Step 2: Form aggregated strings

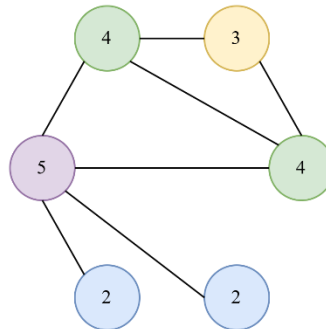
Node	Aggregated String
A	$(1, \{1,1,1\})$
B	$(1, \{1,1\})$
C	$(1, \{1,1,1,1\})$
D	$(1, \{1,1,1\})$
E	$(1, \{1\})$
F	$(1, \{1\})$

Step 3: Hash aggregated strings into new colors

Aggregated String Input	Injective Hash Output
$(1, \{1\})$	2
$(1, \{1,1\})$	3
$(1, \{1,1,1\})$	4
$(1, \{1,1,1,1\})$	5

Thus

Node	New Hashed Color
A	4
B	3
C	5
D	4
E	2
F	2

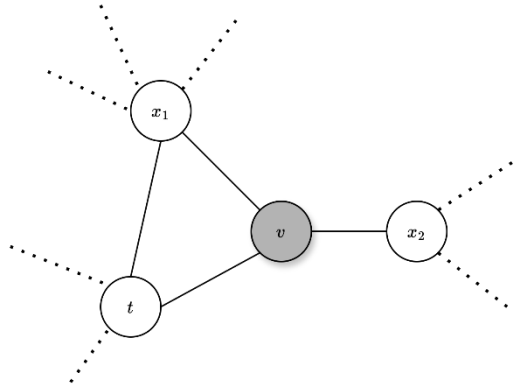


b) A classic and provable example is a triangle (a cycle of 3 nodes).

Specifically, the 1-WL test is completely incapable of distinguishing between:

- A single, continuous hexagon (a cycle of 6 nodes, C_6).
- Two disconnected triangles (two sets of 3-node cycles, $2 \times C_3$).

Question 4: (Node2Vec) (10 points)



Consider an unweighted graph as provided, where a node2vec random walk has just traversed the edge from node t to node v . The walk is currently at node v and needs to decide its next step.

Suppose the node2vec parameters are set as follows:

- Return parameter: $p = 0.5$
- In-out parameter: $q = 2.0$

Given 2nd-order random walk transition probability formula:

$$\pi_{vx} = \alpha_{pq}(t, x) \cdot w_{vx}$$

where $w_{vx} = 1$ in case of unweighted graphs and

$$\alpha_{pq}(t, x) = \begin{cases} \frac{1}{p} & \text{if } d_{tx} = 0 \\ 1 & \text{if } d_{tx} = 1 \\ \frac{1}{q} & \text{if } d_{tx} = 2 \end{cases}$$

- Determine the unnormalized transition probabilities (π) for moving from node v to each of its three neighbors (t , x_1 , and x_2). (5 points)
- How do the parameters p and q allow Node2Vec to interpolate between Breadth-First Search (BFS) and Depth-First Search (DFS) behavior? What specific structural properties of the graph does each search strategy capture? (5 points)

Solution:

a)

$$\pi_{vt} = \frac{1}{p} \cdot 1 = \frac{1}{0.5} = 2$$

$$\pi_{vx_1} = 1 \cdot 1 = 1$$

$$\pi_{vx_2} = \frac{1}{q} \cdot 1 = \frac{1}{2.0} = 0.5$$

b) **Return parameter p :** Controls the likelihood of immediately revisiting a node. A low p encourages the walk to stay local (BFS-like).

In-out parameter q : Controls inward vs. outward exploration. A low ($p < 1$) encourages the walk to venture further away from the origin node (DFS-like).

The Interpolation:

- **BFS-like Behavior (Local):** Achieved by setting a low p and high q . The walk is heavily penalized for moving away from the origin node and is forced to hover in the immediate, local neighborhood.
- **DFS-like Behavior (Global):** Achieved by setting a high p and low q . The walk is penalized for backtracking or staying local, forcing it to explore further out into the network.

Question 5: (Div2Vec) (5 points)

Given a small graph where node A is the current node in a random walk. Node A has three neighbors: X , Y , and Z . The degrees of these neighbor nodes in the overall graph are as follows:

- $\deg(X) = 4$
- $\deg(Y) = 16$
- $\deg(Z) = 16$

Known that, the probability of choosing the next node $v \in N(u)$ defined as:

$$P(v) = \frac{f(\deg(v))}{\sum_{x \in N(u)} f(\deg(x))}$$

and $f(x) = 1/x$

- a) Calculate the exact transition probabilities from node A to nodes X , Y , and Z . (3 points)
- b) What is the theoretical trade-off between using standard div2vec ($f(x) = 1/x$) versus *rooted div2vec* ($f(x) = 1/\sqrt{x}$) in the context of a recommender system? (2 points)

Solution:

a)

First, calculate $f(\deg(v))$ for each neighbor:

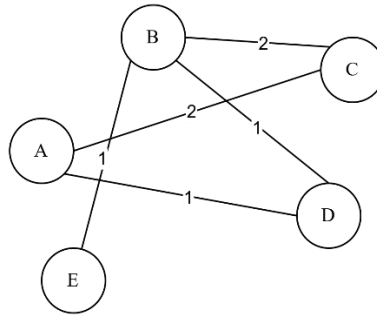
- $f(\deg(X)) = 1/4 = 0.25$
- $f(\deg(Y)) = 1/16 = 0.0625$
- $f(\deg(Z)) = 1/16 = 0.0625$

Then, calculate the probabilities:

- $P(X) = 0.25/0.375 = 2/3$
- $P(Y) = 0.0625/0.375 = 1/6$
- $P(Z) = 0.0625/0.375 = 1/6$

- b) Using $f(x) = 1/x$ strongly penalizes high-degree nodes, which heavily prioritizes diversity but can sometimes decrease overall accuracy (since popular items are often genuinely relevant to many users). Using $f(x) = 1/\sqrt{x}$ softens this penalty, providing a balanced middle ground that improves diversity while maintaining higher predictive accuracy than the standard div2vec approach.

Question 6: (LINE) (10 points)



Given a small, undirected information network with 5 vertices, $V = \{A, B, C, D, E\}$. The connections (edges) between these vertices have specific weights representing the strength of their relationships.

Here are the observed edges and their weights (w):

- Edge (A, C) has a weight of 2
- Edge (A, D) has a weight of 1
- Edge (B, C) has a weight of 2
- Edge (B, D) has a weight of 1
- Edge (B, E) has a weight of 1

Assume all other unlisted vertex pairs have no direct edge, meaning a weight of 0. Also, assume the vertices do not connect to themselves, so $w_{A,A} = 0$.

- What is the **first-order** proximity between vertex A and vertex B? (3 points)
- Given that the second-order proximity relies on the empirical distribution of "contexts" for a given vertex, defined as $\hat{p}_2(v_j|v_i) = \frac{w_{ij}}{d_i}$, where d_i is the degree (sum of edge weights) of vertex i . Calculate the empirical probability $\hat{p}_2(C|A)$ and do vertex A and vertex B have a non-zero second-order proximity? (3 points)
- First-order proximity is only applicable to *undirected graphs*, whereas its formulation for second-order proximity can be applied to *both directed and undirected graphs*. What is the reason? (4 points)

Solution:

a) According to Definition 2 (First-order Proximity) in the paper:

"For each pair of vertices linked by an edge (u, v) , the weight on that edge, w_{uv} , indicates the first-order proximity between u and v . If no edge is observed between u and v , their first-order proximity is 0."

Since there is no direct edge connecting vertex A and vertex B, the local pairwise (first-order) proximity between them is exactly 0.

b) First, we need to calculate the total degree (d_A) for vertex A

$$d_A = w_{AC} + w_{AD} = 2 + 1 = 3$$

Next, we calculate the empirical distribution of context C given vertex A:

$$\hat{p}_2(C|A) = \frac{w_{AC}}{d_A} = \frac{2}{3} \approx 0.667$$

The empirical probability $\hat{p}_2(C|A)$ is $\frac{2}{3}$.

Vertex A and vertex B **have** a non-zero second-order proximity because their neighborhood network structures overlap (they share common neighbors C and D)

c)

First-order proximity is modeled using a joint probability $p_1(v_i, v_j)$ between two vertices.

This probability is calculated using the dot product of their embedded vectors ($\vec{u}_i^T \cdot \vec{u}_j$).

Because a dot product is inherently commutative (symmetric), the joint probability is also symmetric. Therefore, it cannot effectively capture the asymmetric nature of a directed graph (where an edge from A to B does not guarantee an edge from B to A).

Second-order proximity, on the other hand, is modeled using a conditional probability $p_2(v_j|v_i)$ — specifically, the probability of observing context v_j given vertex v_i . Conditional probability is inherently asymmetric, making it well-suited to model directed relationships (e.g., v_i pointing to v_j) found in directed graphs.

Question 7: (HOPE) (10 points)

HOPE model approximates high-order proximities using a general formulation:

$$S = M_g^{-1} \cdot M_l$$

For the Adamic-Adar (AA) measurement, the proximity accounts for the degree of the intermediate nodes. A diagonal matrix D is used, where $D_{ii} = 1/\sum_j (A_{ij} + A_{ji})$. This means D_{ii} is the reciprocal of the total degree (in-degree + out-degree) of vertex v_i . So,

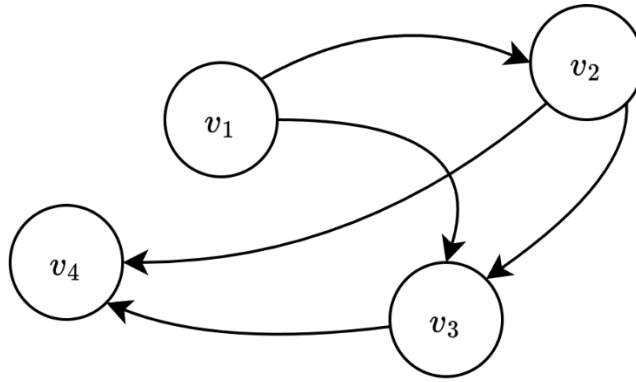
$$\mathbf{M}_g = \mathbf{I}$$

$$\mathbf{M}_l = \mathbf{A} \cdot \mathbf{D} \cdot \mathbf{A}$$

Suppose you are given a directed graph with 4 vertices (v_1, v_2, v_3, v_4) and the following adjacency matrix A :

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

This graph has directed edges $v_1 \rightarrow v_2$, $v_1 \rightarrow v_3$, $v_2 \rightarrow v_3$, $v_2 \rightarrow v_4$, and $v_3 \rightarrow v_4$.



- Based Calculate the total degree for each vertex and construct the diagonal matrix D . (4 points)
- Calculate the exact high-order proximity matrix S for Adamic-Adar, given the adjacency matrix A and matrix D . (4 points)
- Why does the HOPE model avoid directly computing the standard SVD on the high-order proximity matrix S ? In your answer, explain how calculating S explicitly affects matrix density and the overall time complexity compared to the JDGSVD approach. (2 points)

Solution:

a) The diagonal matrix D is:

$$D = \begin{bmatrix} 1/2 & 0 & 0 & 0 \\ 0 & 1/3 & 0 & 0 \\ 0 & 0 & 1/3 & 0 \\ 0 & 0 & 0 & 1/2 \end{bmatrix}$$

b)

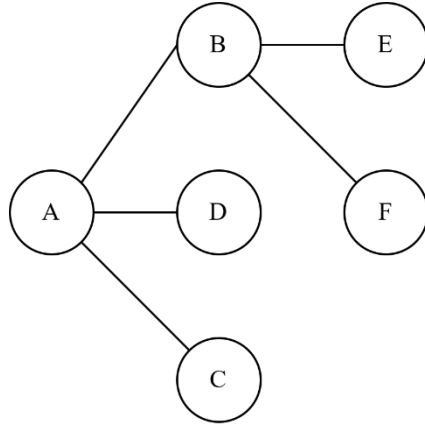
$$\begin{aligned} S = A \cdot D \cdot A &= \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1/2 & 0 & 0 & 0 \\ 0 & 1/3 & 0 & 0 \\ 0 & 0 & 1/3 & 0 \\ 0 & 0 & 0 & 1/2 \end{bmatrix} \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 & 1/3 & 2/3 \\ 0 & 0 & 0 & 1/3 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \end{aligned}$$

c)

- **The Bottleneck of SVD:** Directly computing the standard SVD on S requires explicitly calculating S . Even if the original adjacency matrix A is highly sparse, the matrix polynomial operations required to find S (like matrix inversion or multiplying A by itself multiple times) will make S a very dense matrix.
- **Time Complexity:** Performing matrix inversion and standard SVD on a dense $N \times N$ matrix results in a time complexity of up to $O(N^3)$, which is completely infeasible for large-scale graphs.
- **The JDGSVD Solution:** By using the general formulation $S = M_g^{-1} \cdot M_l$, HOPE never actually computes the dense matrix S . Instead, the JDGSVD algorithm only needs to multiply M_g and M_l with thin $N \times K$ matrices. By multiplying the sparse adjacency matrix A with these thin matrices first, the time complexity drops to $O(m \cdot K^2 \cdot L)$ (where m is the number of edges, K is the embedding dimension, and L is the number of iterations). This makes the time complexity linear with respect to the number of edges, ensuring the model is highly scalable.

Question 8: (struc2vec) (5 points)

Consider an undirected, unweighted graph G with a set of 6 nodes $V = \{A, B, C, D, E, F\}$



Given that

- $R_k(u)$ is the set of nodes at an exact distance of k hops from node u .
- $s(R_k(u))$ is the ordered degree sequence of that set.
- The structural distance at layer k , denoted as $f_k(u, v)$, is defined as:

$$f_k(u, v) = f_{k-1}(u, v) + g\left(s(R_k(u)), s(R_k(v))\right)$$

- Distance function:

$$d(a, b) = \frac{\max(a, b)}{\min(a, b)} - 1$$

- The weight of the edge between u and v in layer k of the context graph is:

$$w_k(u, v) = e^{-f_k(u, v)}$$

- Simplified DTW Calculation (g): When comparing a longer sequence (e.g., $[x, y, z]$) to a sequence of just one number (e.g., $[w]$), calculate the distance by aligning the single number to every number in the longer sequence. The total cost is $d(x, w) + d(y, w) + d(z, w)$.
 - a) What is the structural distance $f_0(A, C)$ and $f_1(A, C)$? (3 points)
 - b) Why do models like DeepWalk or node2vec struggle to identify two nodes as structurally equivalent if they are located far apart in a large network, and how does struc2vec bypass this limitation? (2 points)

Solution:

a) Level 0 Distance ($f_0(A, C)$)

$$f_0(A, C) = \frac{\max(3,1)}{\min(3,1)} - 1$$

$$f_0(A, C) = 3 - 1 = 2$$

Level 1 Distance ($f_1(A, C)$)

Node A's neighbors: Node B (degree 3), Node C (degree 1), Node D (degree 1).

Sorted sequence = 1,1,3

Node C's neighbor: Node A (degree 3). Sorted sequence = [3]

We calculate the DTW cost (g) between the sequences [1,1,3] and [3] using the simplified alignment rule:

- Cost of aligning 1 and 3: $\frac{3}{1} - 1 = 2$
- Cost of aligning the second 1 and 3: $\frac{3}{1} - 1 = 2$
- Cost of aligning 3 and 3: $\frac{3}{3} - 1 = 0$
- Total DTW cost $g = 2 + 2 + 0 = 4$

Finally

$$f_1(A, C) = f_0(A, C) + g$$

$$f_1(A, C) = 2 + 4 = 6$$

b) DeepWalk and node2vec rely on random walks that traverse actual edges in the original graph to generate contexts. If two nodes are further apart than the walk length or the window size, they will never appear in the same context, meaning the model cannot recognize their similarity. struc2vec bypasses this by completely ignoring network proximity. Instead, it builds a brand new "context graph" where edges are formed between nodes based strictly on their mathematical structural similarity, allowing the random walk to jump between identical nodes even if they are on opposite sides of the original network.

Question 9: (Role2Vec) (5 points)

Consider a small graph $G = (V, E)$ consisting of four vertices: $V = \{v_1, v_2, v_3, v_4\}$.

Instead of treating these vertices purely by their IDs, each vertex has an observed attribute/feature vector $x_i \in \mathbb{R}^2$:

$$x_1 = [1, 2]$$

$$x_2 = [3, 0]$$

$$x_3 = [0, 1]$$

$$x_4 = [2, 1]$$

To group these vertices into structural roles (vertex-types), we define a simple mapping function $\Phi(x)$ that uses the sum operator on the attribute vector:

$$\Phi(x_i) = \sum_{k=1}^K x_{i,k}$$

Where $x_{i,k}$ is the k -th element of the attribute vector for vertex i

- Using the mapping function $\Phi(x)$, calculate the resulting vertex-type w for each of the four vertices. (2 points)
- Assume a traditional random walk algorithm traverses the graph and generates the following sequence of vertex IDs: $W = (v_3, v_1, v_4, v_2, v_1)$. Convert this standard random walk into a Role2Vec attributed random walk. (2 points)
- What are the advantages of using *node attributed embeddings* compared to traditional identity-based embeddings? (1 point)

Solution:

a) $w_1 = \Phi(x_1) = 1 + 2 = 3$

$$w_2 = \Phi(x_2) = 3 + 0 = 3$$

$$w_3 = \Phi(x_3) = 0 + 1 = 1$$

$$w_4 = \Phi(x_4) = 2 + 1 = 3$$

Resulting Mappings: $v_1 \rightarrow \text{Type 3}$, $v_2 \rightarrow \text{Type 3}$, $v_3 \rightarrow \text{Type 1}$, $v_4 \rightarrow \text{Type 3}$.

b) Standard Walk: $(v_3, v_1, v_4, v_2, v_1)$

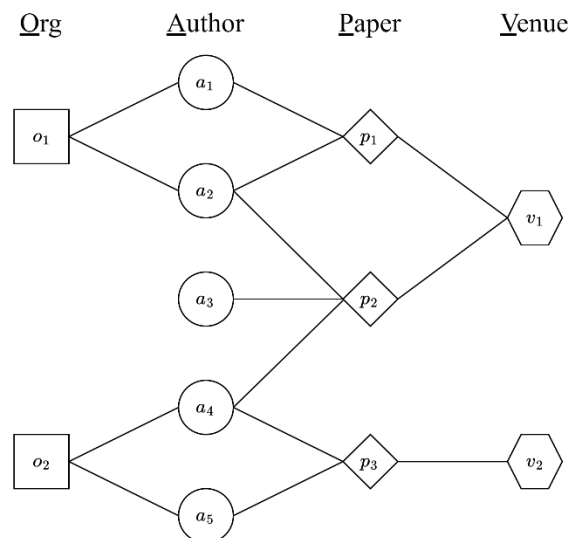
Attributed Walk: $(1, 3, 3, 3, 3)$

c)

- Inductive Learning: By mapping nodes to general "types" based on their features or attributes, the model becomes inductive. If a brand-new node appears, the model simply looks at its attributes, assigns it to the corresponding type, and instantly provides an embedding.
- Better Capturing Structural Roles: Attributed embeddings abstract nodes into feature-based types (e.g., "high-degree hub" or "bridge node"). If two nodes share similar structural features, they are treated identically in an attributed walk, allowing the model to capture their structural similarity (their role) even if they exist on completely opposite sides of the graph.
- Space and Memory Efficiency: Models like Role2Vec only require $O(MD + N_v)$ space, where M is the number of distinct vertex-types. Because the number of unique types is usually vastly smaller than the number of nodes ($M \ll N_v$), the vocabulary size shrinks dramatically, making it highly space-efficient.
- Supports attributed graphs: Provides a mathematical framework to inject this rich metadata directly into the context-generation process, yielding much richer and more informed representations.

Question 10: (metapath2vec) (5 points)

Consider the following heterogeneous academic network:



- List all valid APVPA meta-paths that start with author a_2 and end with a different author. (3 points)
- Meta-paths are typically designed to be symmetric (where the first node type is the same as the last node type, such as APVPA). What is the algorithmic necessity of this symmetry when generating paths for the skip-gram model? (2 points)

Solution:

- Valid APVPA meta-paths starting with a_2 and ending with a different author
 - $a_2 \rightarrow p_1 \rightarrow v_1 \rightarrow p_1 \rightarrow a_1$
 - $a_2 \rightarrow p_1 \rightarrow v_1 \rightarrow p_2 \rightarrow a_3$
 - $a_2 \rightarrow p_1 \rightarrow v_1 \rightarrow p_2 \rightarrow a_4$
 - $a_2 \rightarrow p_2 \rightarrow v_1 \rightarrow p_1 \rightarrow a_1$
 - $a_2 \rightarrow p_2 \rightarrow v_1 \rightarrow p_2 \rightarrow a_3$
 - $a_2 \rightarrow p_2 \rightarrow v_1 \rightarrow p_2 \rightarrow a_4$
- Symmetry allows the random walk to be continuous and recursive. Because the sequence ends on the exact same node type it started on, the walker can seamlessly restart the meta-path template, generating infinitely long paths without getting "stuck" on an incompatible node type.

Question 11: (HIN2Vec) (10 points)

Given a small Heterogeneous Information Network (HIN) with a total of 4 nodes ($|V| = 4$). We want to embed these nodes into a 2-dimensional latent space ($d = 2$). The model is set up to predict 3 different meta-path relationships ($|R| = 3$).

Suppose the trained transformation matrix for the source node, W_X , is given as:

$$W_X = \begin{bmatrix} 0.1 & 0.2 \\ 0.5 & 0.9 \\ 0.4 & 0.6 \\ 0.8 & 0.3 \end{bmatrix}$$

- What are the dimensions of the input vector $\vec{x}$, the transformation matrix W_X , and its transpose W_X' ? Write out the one-hot encoded vector $\vec{x}$. (3 points)
- According to the HIN2Vec framework, the latent vector for node x is computed as $W_X'\vec{x}$. Calculate this resulting latent vector step-by-step. (3 points)
- During the training data preparation phase of HIN2Vec, why is it crucial to explicitly filter out cycles (paths where the same node appears more than once) when generating random walks? (4 points)

Solution:

- Dimensionality

The one-hot vector $\vec{x}$ has dimensions 4×1 .

W_X : 4×2 matrix.

W_X' : The transpose of W_X is 2×4 matrix

- Node x is the third node (index 2 in a 0-indexed system), its one-hot vector has a 1 in the third position and 0s everywhere else:

$$\vec{x} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

First, we find the transpose W_X' :

$$W_X' = \begin{bmatrix} 0.1 & 0.5 & 0.4 & 0.8 \\ 0.2 & 0.9 & 0.6 & 0.3 \end{bmatrix}$$

Next,

$$W_X'\vec{x} = \begin{bmatrix} 0.1 & 0.5 & 0.4 & 0.8 \\ 0.2 & 0.9 & 0.6 & 0.3 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

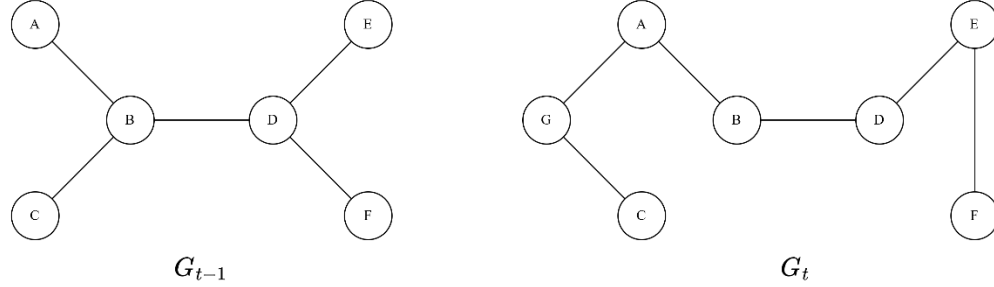
$$W_X'\vec{x} = \begin{bmatrix} (0.1 \times 0) + (0.5 \times 0) + (0.4 \times 1) + (0.8 \times 0) \\ (0.2 \times 0) + (0.9 \times 0) + (0.6 \times 1) + (0.3 \times 0) \end{bmatrix} = \begin{bmatrix} 0.4 \\ 0.6 \end{bmatrix}$$

- Cycles must be eliminated because they generate training samples that violate the real-world semantics of the targeted relationships. For instance, if a random walk loops back on itself through the A-P-A meta-path (e.g., traversing Author 1 $\rightarrow$ Paper 3 $\rightarrow$ Author 1), it generates

the training triple $\langle A_1, A_1, A-P-A \rangle$. This implies a "self-coauthorship," which is semantically invalid and would corrupt the model's ability to accurately learn the true nature of collaborative relationships.

Question 12: (dynnode2vec) (10 points)

Consider a dynamic network that evolves from timestamp $t - 1$ to timestamp t as follow:



- Identify the set of newly added nodes (V_{add}), set of newly added edges (E_{add}) and the set of deleted edges (E_{del}). (5 points)
- According to the dynnode2vec architecture, how exactly is the set of 'evolving nodes' (ΔV_t) defined, and under what conditions would the time efficiency of this generation step degrade to match that of static node2vec? (5 points)

Solution:

a)

Newly added nodes (V_{add}): The only new node introduced at timestamp t is G.

$$V_{add} = \{G\}$$

Newly added edges (E_{add}): These are edges present in G_t but not in G_{t-1} .

$$E_{add} = \{(A, G), (B, C), (D, F), (E, F)\}$$

Deleted edges (E_{del}): These are edges present in G_{t-1} but no longer present in G_t .

$$E_{del} = \{(B, D), (D, E)\}$$

- In the dynnode2vec architecture, the generation of new random walks is restricted to regions of the graph that have experienced structural changes to save computation time. The set of "evolving nodes" (ΔV_t) is formally defined as the union of all newly added nodes and any existing nodes that act as endpoints for newly added or deleted edges.

Mathematically, this is expressed as:

$$\Delta V_t = V_{add} \cup \{v_i \in V_t | \exists e_i = (v_i, v_j) \in (E_{add} \cup E_{del})\}$$

The time efficiency of dynnode2vec's random walk generation step degrades to match that of static node2vec under the worst-case scenario where every node in the network qualifies as an evolving node (i.e., $\Delta V_t = V_t$). In this case, the algorithm is forced to generate new random walks for all nodes.

Question 13: (Continuous-Time Dynamic Network Embeddings) (5 points)

Suppose you are applying the CTDNE framework to a small, continuous-time dynamic network $G = (V, E_T, \mathcal{T})$. The network contains exactly three temporal edges that occurred over a short timeframe:

Edge 1 (e_1): Node A interacts with Node B at timestamp $\mathcal{T}(e_1) = 1$

Edge 2 (e_2): Node B interacts with Node C at timestamp $\mathcal{T}(e_2) = 2$

Edge 3 (e_3): Node A interacts with Node C at timestamp $\mathcal{T}(e_3) = 3$

- a) Using the exponential biased distribution, calculate the probability of selecting e_2 as the initial edge. *Note: You can leave your final answer as a fraction in terms of Euler's number (e).* Given the Exponential Biased Selection Formula: (3 points)

$$\Pr(e) = \frac{\exp(\mathcal{T}(e) - t_{min}))}{\sum_{e' \in E_T} \exp(\mathcal{T}(e') - t_{min}))}$$

- b) When choosing the very first edge to start a temporal random walk, how does applying the exponential bias distribution alter the behavior of the model compared to the unbiased (uniform) approach? (2 points)

Solution:

- a) Here, $t_{min} = 1$.

Calculate the exponential weight for each edge:

- Weight for e_1 : $\exp(1 - 1) = e^0 = 1$
- Weight for e_2 : $\exp(2 - 1) = e^1 = e$
- Weight for e_3 : $\exp(3 - 1) = e^2$

$$\Pr(e_2) = \frac{e}{1 + e + e^2}$$

- b) The unbiased approach treats all interactions equally, regardless of when they occurred. The exponential bias heavily favors starting walks on edges that appeared later in time, under the assumption that more recent network dynamics are often more predictive of the network's current or future state.